

Module 5: Critical Thinking Assignment

CSC450-1 Programming III, Colorado State University – Global Campus

Professor H. Pensado

User Input Program

Hugo Martinez

March 13, 2026

User Input Program

This assignment focuses on demonstrating core C++ programming concepts related to file input/output operations, user input handling, and string manipulation. The objective of the program is to accept input from the user and append that data to an existing text file without overwriting the file's original contents. This requirement emphasizes the use of file stream operations and append mode to ensure that previously stored information remains intact. In addition to appending user input to the provided file, the program also includes a reversal method that reads the contents of the file, reverses all characters contained within it, and writes the reversed output to a separate text file.

Several key programming concepts are explored, including user input processing, file handling using input and output streams, function implementation, and basic algorithmic operations such as reversing a sequence of characters. Together, these elements illustrate how C++ programs can interact with external data sources while maintaining structured and readable code.

Pseudocode and Program Design

The following pseudocode describes how the program will first collect input from the user and append that input to the provided text file. After the data has been successfully appended, the program then reads the entire contents of the file and processes the text by reversing all characters. The reversed text is subsequently written to a separate output file. The use of functions in the pseudocode also reflects the modular structure of the final program, separating the responsibilities of appending data and reversing file contents.

START

Declare string variable userInput

Prompt the user to enter text

Store the input in userInput

CALL appendToFile(userInput)

CALL reverseFile()

Display message "Program completed successfully"

END

```
FUNCTION appendToFile(text)
```

```
Open CSC450_CT5_mod5.txt in append mode
```

```
IF file opens successfully
```

```
    Write text to file
```

```
    Close file
```

```
ELSE
```

```
    Display error message
```

```
END IF
```

```
END FUNCTION
```

```
FUNCTION reverseFile()
```

```
Open CSC450_CT5_mod5.txt for reading
```

```
Read contents of file into a string variable
```

```
Close file
```

```
Reverse all characters in the string
```

```
Open CSC450-mod5-reverse.txt for writing
```

```
Write reversed string to file
```

```
Close file
```

```
END FUNCTION
```

The application begins by prompting the user to enter text that will be appended to the provided file, *CSC450_CT5_mod5.txt*. To ensure that the original file contents are preserved, the program opens the file using append mode.

Once the input has been written to the file, the program reads the entire contents of the file and stores it within a string variable. The stored text is then reversed using a standard string manipulation approach. The reversed content is subsequently written to a new file named *CSC450-mod5-reverse.txt*.

To improve readability and organization, the program separates functionality into two functions: one responsible for appending user input to the file and another responsible for reversing the file contents.

```
#include <iostream>
#include <fstream>
#include <string>
#include <algorithm>

using namespace std;

// Function to append user input to the file
void appendToFile(string text)
{
    ofstream outFile("CSC450_CT5_mod5.txt", ios::app);

    if (outFile.is_open())
    {
        outFile << text << endl;
        outFile.close();
        cout << "Input successfully appended to the file." << endl;
    }
    else
    {
        cout << "Error: Unable to open file for writing." << endl;
    }
}

// Function to reverse file contents and store in a new file
```

```
void reverseFile()
{
    ifstream inFile("CSC450_CT5_mod5.txt");
    string fileContent = "";
    string line;

    if (inFile.is_open())
    {
        while (getline(inFile, line))
        {
            fileContent += line + "\n";
        }
        inFile.close();
    }
    else
    {
        cout << "Error: Unable to open file for reading." << endl;
        return;
    }

    // Reverse all characters
    reverse(fileContent.begin(), fileContent.end());

    ofstream reverseFile("CSC450-mod5-reverse.txt");

    if (reverseFile.is_open())
    {
        reverseFile << fileContent;
        reverseFile.close();
        cout << "Reversed content written to CSC450-mod5-reverse.txt" << endl;
    }
    else
    {
        cout << "Error: Unable to create reverse file." << endl;
    }
}
```

```
    }  
}  
  
int main()  
{  
    string userInput;  
  
    cout << "Enter text to append to the file: ";  
    getline(cin, userInput);  
  
    // Call function to append text  
    appendToFile(userInput);  
  
    // Call function to reverse file contents  
    reverseFile();  
  
    cout << "Program completed successfully." << endl;  
  
    return 0;  
}
```

Program Execution and Results

When the program runs, the user is prompted to enter text that will be appended to the provided file. After the text is successfully appended, the program reads the entire contents of the file, reverses the characters, and writes the reversed output to a new file.

The following screenshots demonstrate the successful execution of the program, including the console output, the updated input file, and the generated reverse output file.

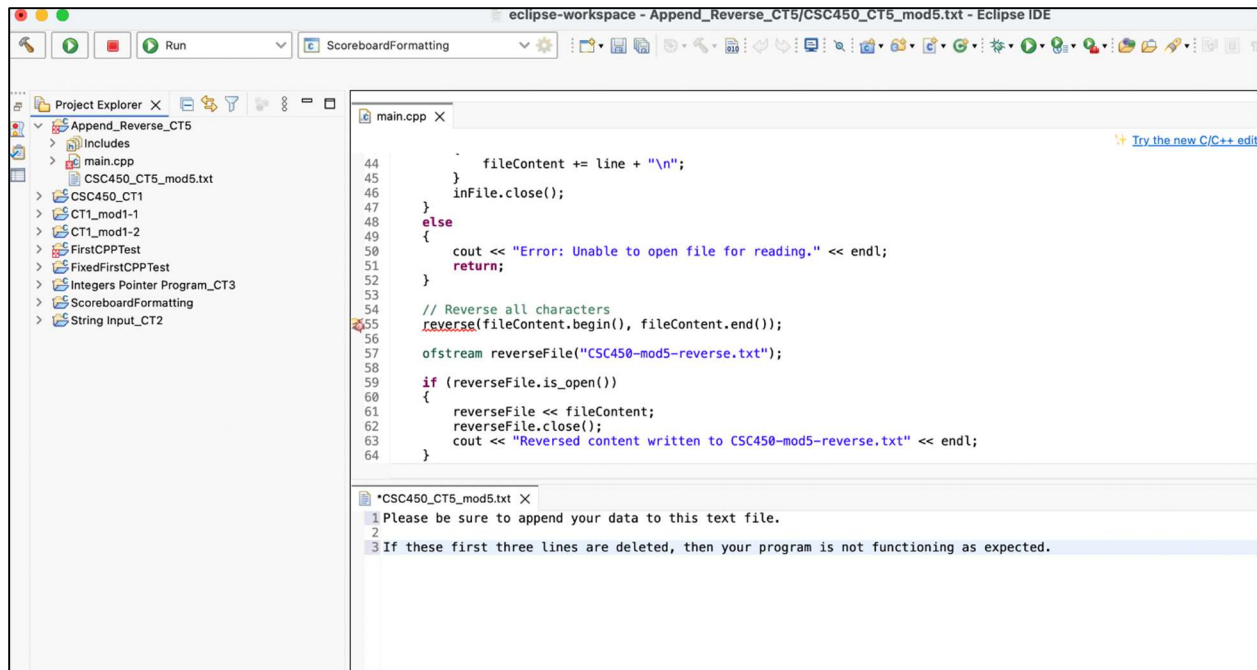


Figure 1. Program execution recognizing file input.

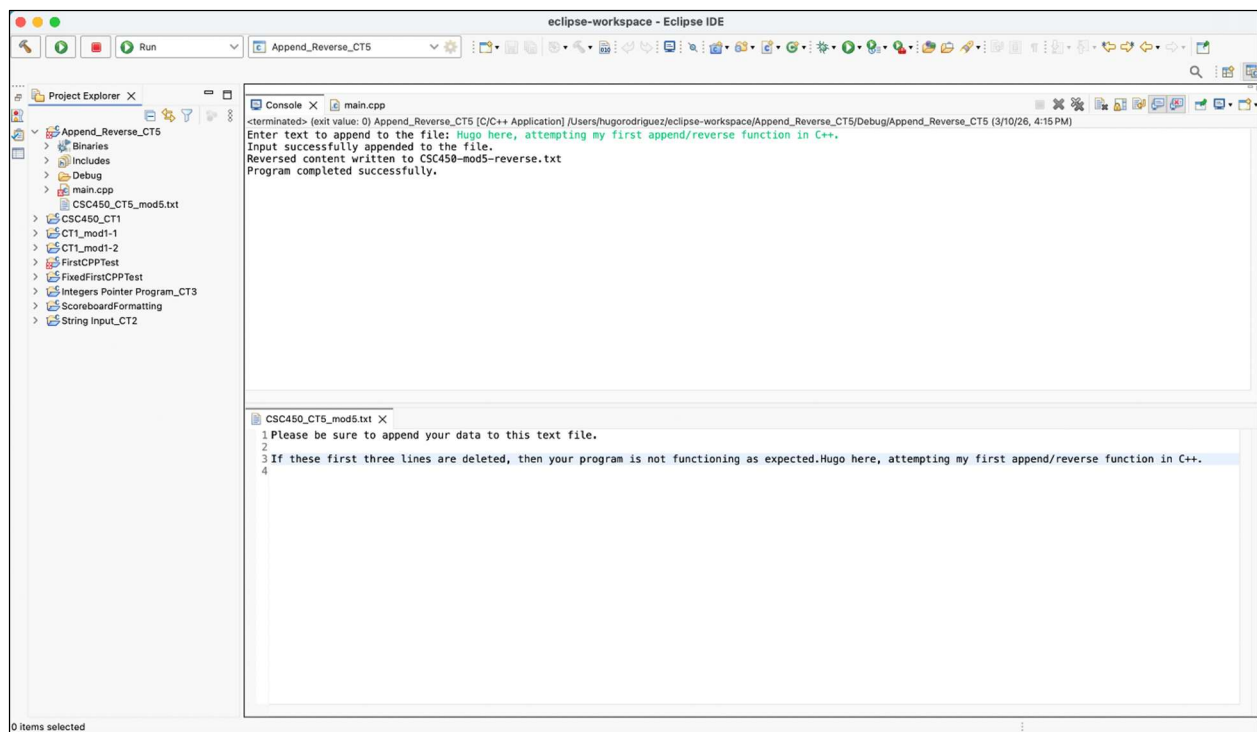


Figure 2. Console showing successful user input, file append and reverse file creation.

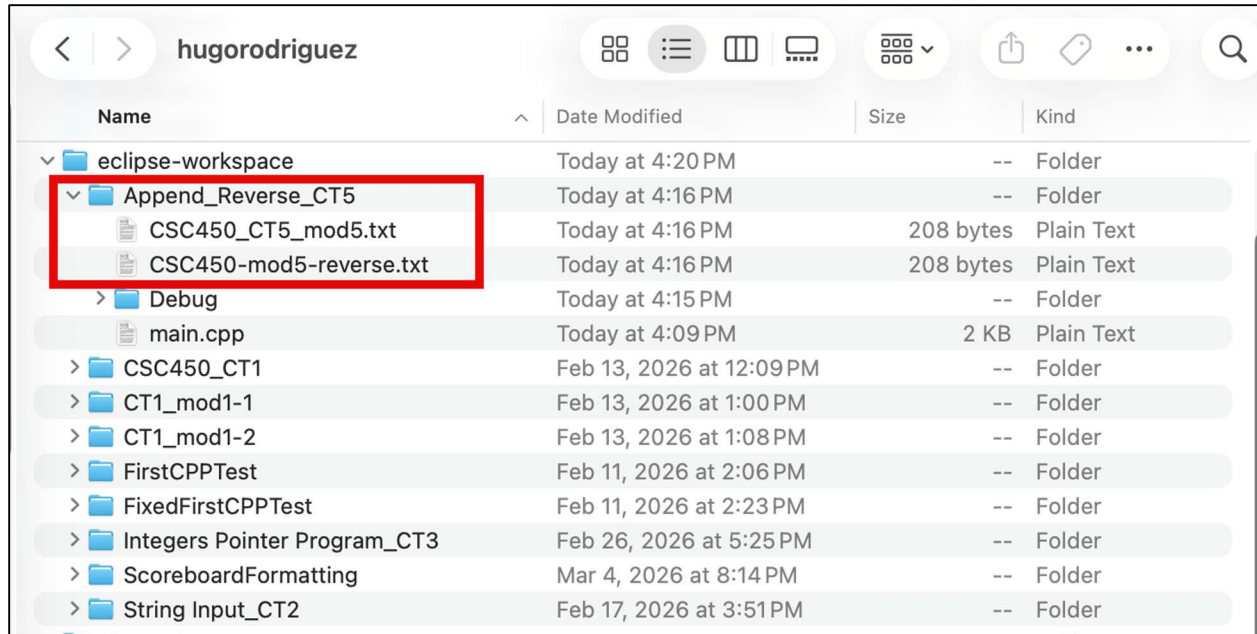


Figure 3. Project directory displaying both original input file and reverse output created by the program.

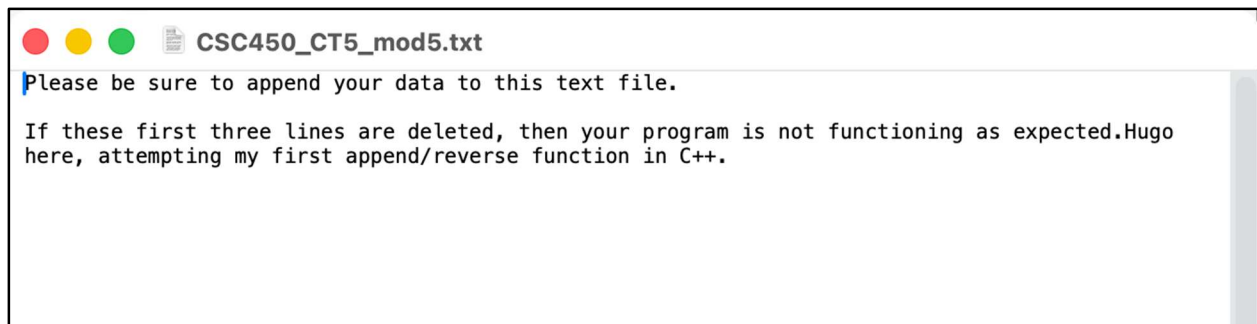


Figure 4. Contents of CSC450_CT5_mod5.txt file with appended input.

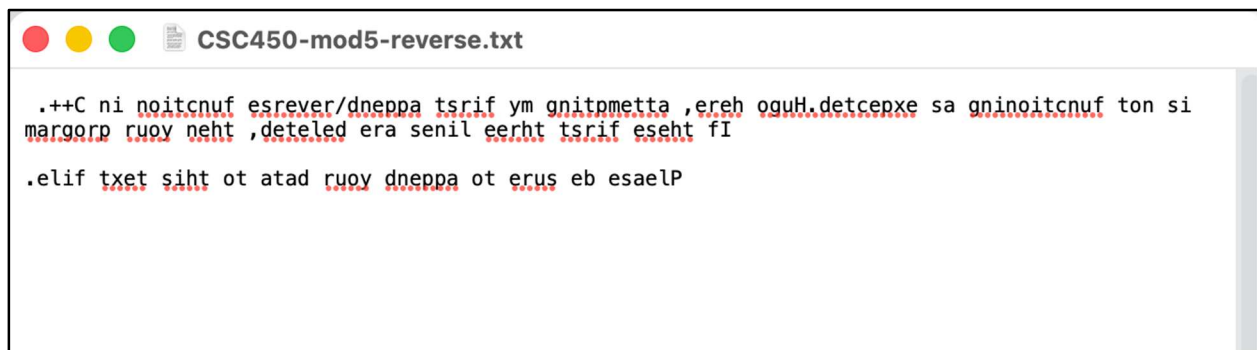


Figure 5. Contents of CSC450-mod5-reverse.txt file showing reversed characters from original file.

Program Analysis

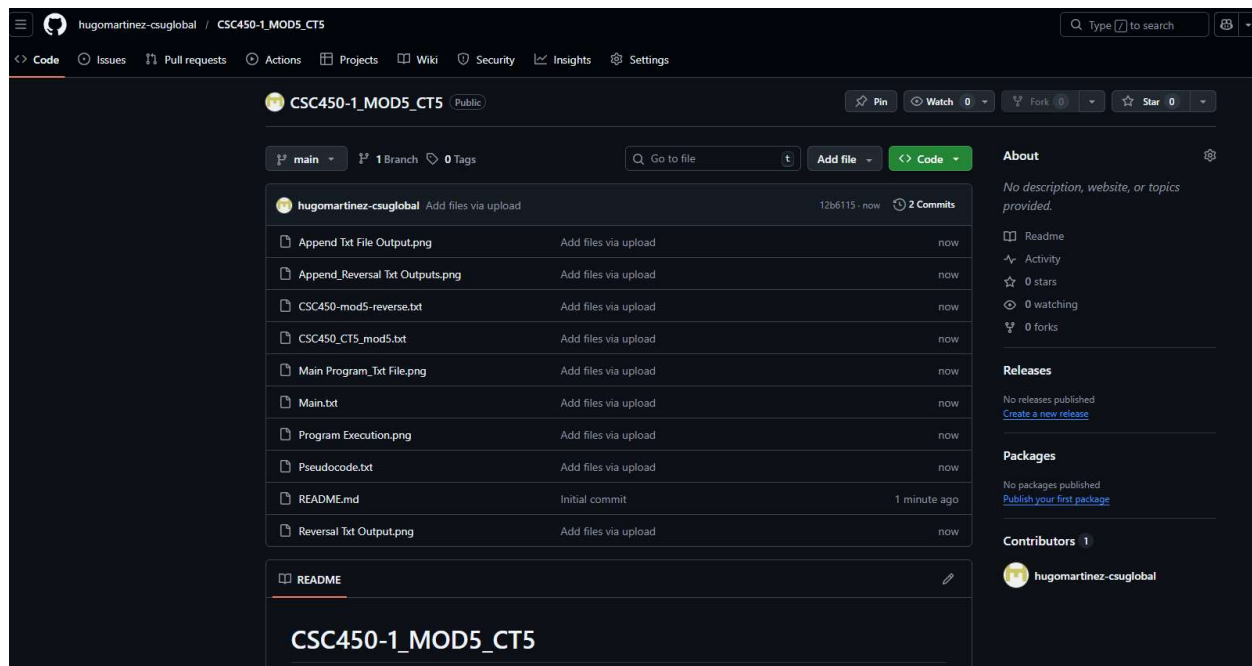
Although the program performs its intended functionality correctly, there are several potential security considerations associated with accepting user input and writing data to files. One potential vulnerability is the lack of input validation. Since the program accepts any text entered by the user, it could allow extremely long input strings or unexpected characters to be written to the file, which may cause performance issues or unintended behavior in larger applications.

Another concern relates to file access and error handling. If the file cannot be opened due to incorrect permissions, missing files, or directory access restrictions, the program may fail to operate as intended. While the program includes basic checks to verify whether files are successfully opened, more robust error handling could be implemented in a production environment to ensure that failures are handled gracefully.

To reduce these vulnerabilities, input validation techniques could be implemented to limit the length or format of user input before writing it to the file. Additionally, stronger exception handling and logging mechanisms could be introduced to detect and respond to file access errors more effectively. Implementing these safeguards would make the program more reliable and secure when handling user-provided data.

GitHub Repository

The submission details can be found at the following GitHub Repository link:
https://github.com/hugomartinez-csuglobal/CSC450-1_MOD5_CT5



Conclusion

In this assignment, a C++ program was developed to demonstrate fundamental programming concepts related to user input, file handling, and string manipulation. The program successfully accepts input from the user and appends the data to an existing text file without overwriting the file's original contents. This requirement highlights the use of file output streams and append mode to preserve previously stored data.

The program also implements a reversal method that reads the contents of the file, reverses all characters within the text, and writes the resulting output to a separate file. By separating these tasks into dedicated functions, the program maintains a modular and organized structure that improves readability and maintainability. These concepts form an important foundation for building more complex applications that interact with external data sources and process user-provided information.